Лабораторная работа №1

«Цветовые модели»

Дисциплина: Компьютерная графика

Выполнил: студент Никончик Виктор Александрович

курс 3, группа 3

1 Цель работы

Изучить принципы построения и математические основы цветовых моделей RGB, HSV (HSB) и CMYK.

Создать десктопное приложение, позволяющее пользователю выбирать цвет на цветовом круге, задавать его численно в моделях RGB, HSV и CMYK и видеть при этом составляющие цвета во всех моделях одновременно. Сравнить собственную реализацию с готовым компонентом выбора цвета из сторонней библиотеки.

2 Математика преобразования моделей

Все преобразования выполняются через базовое пространство RGB с нормализацией компонент к диапазону [0, 1]. В интерфейсе значения отображаются в следующих единицах: RGB — 0…255, тон H — 0…359°, S, V и CMYK — в процентах.

Приведённые формулы CMYK описывают идеализированную модель (чистые краски, белая бумага). Реальный результат печати зависит от ICC-профиля конкретного устройства, бумаги и драйвера, поэтому значения CMYK в программе следует рассматривать как приближённые.

2.1 RGB ⇔ CMYK

$$K = 1 - max(R,\ G,\ B)$$

$$C = M = Y = 0 \quad при\ K = 1$$

$$C = \frac{1 - R - K}{1 - K}, \quad M = \frac{1 - G - K}{1 - K}, \quad Y = \frac{1 - B - K}{1 - K} \quad при\ K < 1$$

$$R = (1 - C)(1 - K), \quad G = (1 - M)(1 - K), \quad B = (1 - Y)(1 - K)$$

Прямое преобразование выбирает единственное разложение с максимальным K, тогда как обратное допускает любые сочетания: например, C = M = Y = 50 %, K = 0 и C = M = Y = 0, K = 50 % дают один и тот же серый цвет. Поэтому при вводе CMYK введённые значения не заменяются пересчитанными, чтобы поля не «прыгали» под руками пользователя.

2.2 RGB ⇔ HSV

$$M = max(R,\ G,\ B), \quad m = min(R,\ G,\ B), \quad \Delta = M - m$$

$$V = M, \qquad S = \frac{\Delta}{M} \quad (S = 0 \ \ при \ \ M = 0)$$

$$H = 60° \cdot \left(\frac{G - B}{\Delta} \bmod 6 \right) \quad \text{при } M = R$$

$$H = 60° \cdot \left(\frac{B - R}{\Delta} + 2 \right) \quad \text{при } M = G$$

$$H = 60° \cdot \left(\frac{R - G}{\Delta} + 4 \right) \quad \text{при } M = B$$

При Δ = 0 (оттенки серого) тон не определён. В собственной реализации в этом случае сохраняется предыдущее значение H, чтобы маркер на круге не «прыгал» при вводе серого цвета.

Обратное преобразование HSV ⇒ RGB:

$$C = V \cdot S, \quad X = C \cdot \left(1 - \left| \left(\frac{H}{60°} \right) \bmod 2 - 1 \right| \right), \quad m = V - C$$

$$(R_1,\ G_1,\ B_1) = (C, X, 0),\ (X, C, 0),\ (0, C, X),\ (0, X, C),\ (X, 0, C),\ (C, 0, X)$$

для H из диапазонов [0°, 60°), [60°, 120°), …, [300°, 360°) соответственно; итоговый цвет (R, G, B) = (R_1 + m, G_1 + m, B_1 + m). В программе прямое и обратное преобразования выполняются стандартными методами Color.RGBtoHSB и Color.getHSBColor.

2.3 Геометрия цветового круга

Собственный компонент состоит из кольца тона и квадрата насыщенности/яркости, вписанного во внутренний круг. Пусть (cx, cy) — центр, (x, y) — точка нажатия мыши, a — половина стороны квадрата.

Тон определяется углом точки на кольце (ось Y экрана направлена вниз, поэтому берётся −dy, и угол отсчитывается против часовой стрелки от направления «на 3 часа»):

$$dx = x - cx, \quad dy = y - cy, \quad H = \operatorname{atan2}(-dy,\ dx) \bmod 360°$$

Насыщенность и яркость определяются положением точки в квадрате и ограничиваются отрезком [0, 1]:

$$S = \frac{dx + a}{2a}, \qquad V = 1 - \frac{dy + a}{2a}, \qquad a = \frac{r}{\sqrt{2}}$$

где r — радиус внутреннего круга. Квадрат закрашивается двумя градиентами: по горизонтали от белого к чистому тону (S: 0 → 1) и поверх него по вертикали от прозрачного к чёрному (V: 1 → 0).

3 Параметры окружения и используемый стек

- Язык и стандарт: Java 21
- Графический фреймворк: Java Swing / AWT (Graphics2D, GradientPaint)
- Стандартное решение: библиотека org.drjekyll:colorpicker 2.0.1
- Система сборки: Maven
- Целевая платформа: любая ОС с JRE 21

4 Интерфейс и реализация приложения

Главное окно разделено на две равные части: слева — собственная реализация выбора цвета (ColorPickerCustom), справа — стандартный компонент из библиотеки (ColorPickerStd).

Собственная реализация (класс ColorPickerCustom, наследник JPanel) включает:

- кольцо тона H и вписанный в него квадрат S/V; выбор цвета нажатием и перетаскиванием мыши; режим перетаскивания (кольцо или квадрат) фиксируется в момент нажатия;
- маркеры текущего цвета на кольце и в квадрате, которые вычисляются из значений H, S, V при каждой перерисовке и потому корректно ведут себя при изменении размеров окна;
- поля численного ввода H, S, V, R, G, B и C, M, Y, K; изменение любого поля пересчитывает остальные и обновляет круг, а флаг updating предотвращает циклические обновления.

Стандартная реализация (класс ColorPickerStd) использует готовый компонент ColorPicker, который сам содержит поля ввода RGB и HSB. Он дополнен плашкой с текущим цветом и полями ввода C, M, Y, K: при вводе CMYK цвет передаётся в компонент методом setColor, а при выборе цвета в компоненте поля CMYK пересчитываются автоматически.

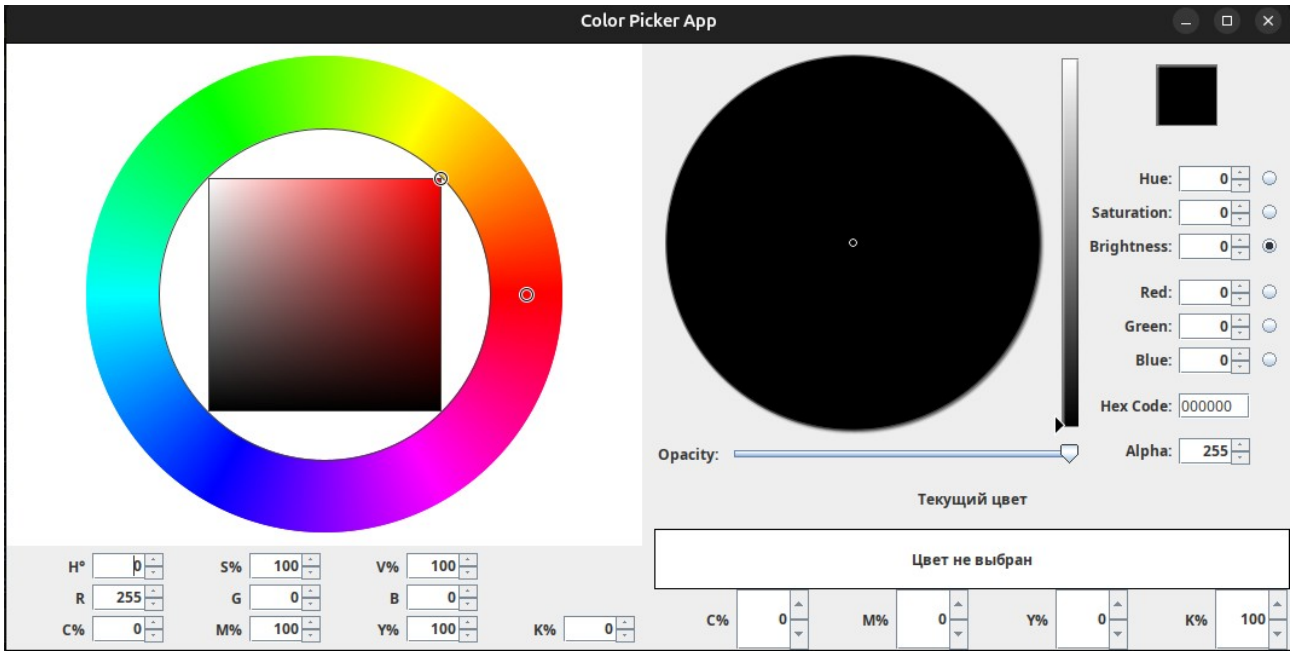


Рис. 1: Главное окно программы: собственная (слева) и стандартная (справа) реализации выбора цвета

5 Выводы

В ходе выполнения работы было разработано десктопное приложение для выбора цвета, изучены цветовые модели RGB, HSV и CMYK и преобразования между ними, а также:

- Реализован собственный компонент выбора цвета на цветовом круге с квадратом насыщенности/яркости.
- Для собственной реализации доступны следующие способы задания цвета:
  - мышью на кольце тона и в квадрате S/V;
  - численным вводом в моделях HSV, RGB и CMYK.
- При изменении цвета любым способом все остальные значения (HSV, RGB, CMYK) автоматически пересчитываются.
- Обе реализации — собственная и стандартная — дополнены полями ввода CMYK с автоматическим пересчётом остальных моделей.
- Проведено сравнение собственной реализации с готовым библиотечным компонентом.

6 Ссылки

GitHub-репозиторий проекта: graphics-color-picker